

Audio RAG

Hybrid Search Over Audio Transcripts

Keyword + Semantic Search | Speaker Diarization | Reciprocal Rank Fusion

The Problem

- Audio recordings contain valuable information but are **unsearchable by default**
- Transcripts alone aren't enough — users search using **their own words**, not the exact words spoken
- Need to know **who said what** and **when** — not just find matching text
- Keyword search fails when query wording differs from transcript wording

Challenge: Build a hybrid search system that handles both exact keyword queries and natural-language meaning queries across multi-speaker audio recordings.

Our Solution

A four-stage pipeline: **Transcribe** → **Diarize** → **Index** → **Search**

Pipeline

- **Transcribe** audio with word-level timestamps
- **Identify speakers** using voice fingerprinting
- **Align** words to speakers, chunk by speaker turn
- **Embed + Index** into Postgres with pgvector
- **Search** with keyword + semantic + RRF fusion

Every result returns

File: andrew_ng_ai_opportunities.wav

Speaker: Andrew Ng

Time: 02:04 – 02:28

Match: keyword + semantic

Text: "The university system is slow to adapt..."

Architecture

Audio Files (5 × 10 min, 2 speakers each)

|



faster-whisper (local) → words + timestamps

pyannote.audio (local) → speaker segments

|



Alignment + Chunking → speaker turns

|



all-MiniLM-L6-v2 (local) → 384-dim embeddings

Postgres tsvector → full-text tokens

|



Postgres + pgvector (Docker) → unified storage

|



Hybrid Search (RRF) → ranked results

Key Design Decisions

Decision	Choice	Rationale
Chunking	By speaker turn	Natural boundary, preserves speaker attribution
Embedding model	all-MiniLM-L6-v2 (384-dim)	Fast on CPU, strong quality
Search fusion	Reciprocal Rank Fusion	Simple, effective, no weight tuning
Database	Postgres + pgvector	Vectors + full-text in one DB
Transcription	faster-whisper (local)	Free, accurate, word-level timestamps
Diarization	pyannote.audio (local)	State-of-the-art open-source

Why Hybrid Search?

Keyword search alone fails 61% of the time.

User searches:

"what caused the fall of Constantinople"

Transcript says:

"armies from mostly France sack
Constantinople and dismember the
empire"

**No exact word overlap → keyword
returns NOTHING**

Semantic search:

Understands "caused the fall" ≈ "sack and
dismember"

Finds the right chunk ✓

Hybrid (RRF):

Boosts results that match on *both*
keywords and meaning

Best ranking ✓

Evaluation Design

36 golden queries across 3 categories, **43 expected results**, tested against 3 search strategies

Category	Count	Purpose	Example
keyword	12	Exact words from transcript	"Sam Altman accused of murder"
semantic	15	Different words, same meaning	"are we living in a computer simulation"
hybrid	8	Partial keyword + meaning	"Epstein did not kill himself guards cameras"

8 multi-result queries test cross-segment and cross-file retrieval

Metrics: Recall@k (did we find it?), MRR (how high did we rank it?), Precision@k (how much noise?)

Results

0.957

Recall@10
Target ≥ 0.85 ✓

0.858

MRR
Target ≥ 0.60 ✓

0.929

Recall@5
Target ≥ 0.70 ✓

Strategy	Recall@5	Recall@10	MRR
Hybrid	0.929	0.957	0.858
Keyword Only	0.386	0.386	0.400
Semantic Only	0.929	0.957	0.772

Per-Category Breakdown

This is where hybrid search proves its value.

Strategy	keyword (12)	semantic (15)	hybrid (8)
Hybrid Recall@10	0.917	0.967	1.000
Keyword Recall@10	0.625	0.133	0.500
Semantic Recall@10	0.917	0.967	1.000

- Keyword search scores **0.133 recall on semantic queries** — near total failure
- Hybrid achieves **perfect recall + MRR on hybrid-category queries**
- Semantic drives recall; keyword improves ranking via RRF

Key Observations

1. **Keyword search alone is insufficient** — returned NO RESULTS for 22 out of 35 queries (61%)
2. **Semantic search is the primary driver of recall** — hybrid and semantic have identical recall (0.957)
3. **Hybrid's advantage is ranking (MRR)** — RRF boosts results matching both modalities, improving MRR by 11% over semantic-only (0.858 vs 0.772)
4. **Multi-result queries are harder** — recall drops to 0.5 at k=5 when expecting 2+ results
5. **One persistent failure** — "condolence cards" query fails due to chunking boundary, not search

Limitations & Future Work

Limitations

- 36 queries / single annotator — directional, not statistically robust
- Precision@k structurally low with 1-2 expected results per query
- Transcription errors propagate to both search modalities
- CPU-only: diarization takes ~15 min per 10-min file

Future Improvements

- Overlapping chunks for better boundary coverage
- Cross-encoder re-ranking for higher precision
- GPU acceleration for real-time indexing
- User feedback loop to improve ranking
- Multiple annotators for golden dataset

Tech Stack

Component	Tool	Runs
Transcription	faster-whisper	Local, free
Diarization	pyannote.audio 3.1	Local, free
Embeddings	all-MiniLM-L6-v2	Local, free
Database	Postgres 16 + pgvector	Docker
Keyword search	Postgres tsvector/tsquery	Built-in
Semantic search	pgvector cosine similarity	Built-in
Search fusion	Reciprocal Rank Fusion	Python

Fully local. No paid APIs. Reproducible.